



IA CORPORATIVA COM JAVA E LANGCHAIN4J

AULA 05 - INGESTÃO DE DADOS COM EASYRAG



OBJETIVOS DA AULA

- ❑ Dissecar a "mágica" da extensão `quarkus-langchain4j-easy-rag`, entendendo os componentes que ela configura automaticamente.
- ❑ Aprofundar na configuração do `DocumentSplitter`, analisando o trade-off técnico entre tamanho do segmento e sobreposição.
- ❑ Otimizar a ingestão de documentos ajustando `max-segment-size` e `max-overlap-size` como decisões de engenharia.
- ❑ Utilizar a Dev UI do Quarkus como ferramenta de depuração para inspecionar o `InMemoryEmbeddingStore` e validar a estratégia de segmentação.



DESMISTIFICANDO A "MÁGICA" DO EASYRAG



DESMISTIFICANDO A "MÁGICA" DO EASYRAG

- ❑ Anteriormente, nós adicionamos a extensão easy-rag, apontamos para um diretório e nosso agente "aprendeu" sobre nossos pacotes de viagem.
- ❑ O que a Extensão Faz nos Bastidores?
 - ❑ A extensão quarkus-langchain4j-easy-rag é um exemplo de “convention over configuration” que executa um pipeline de ingestão completo na inicialização da aplicação:
 1. Criação do EmbeddingStore Padrão: A extensão verifica se algum bean do tipo EmbeddingStore (como um PgVectorEmbeddingStore ou QdrantEmbeddingStore) foi configurado por nós. Como não configuramos nenhum, ela assume que estamos em modo de prototipagem e instancia um InMemoryEmbeddingStore.
 2. Execução do Pipeline de Ingestão:
 - A. Load: Ela carrega todos os arquivos do diretório especificado em quarkus.langchain4j.easy-rag.path.
 - B. Split: Utiliza um DocumentSplitter com configurações padrão para quebrar os documentos em chunks.
 - C. Embed & Store: Para cada chunk, ela usa o EmbeddingModel default (nomic-embed-text via Ollama) para gerar um embedding e o armazena, junto com o texto original, no InMemoryEmbeddingStore.
 3. Injeção Automática do RetrievalAugmentor: A extensão cria um bean RetrievalAugmentor pré-configurado para usar o InMemoryEmbeddingStore. O Quarkus então detecta que nosso AiService (TravelAgentAssistant) pode ser aprimorado por um RetrievalAugmentor e o injeta automaticamente no pipeline de execução do serviço.
- ❑ É por isso que nosso código Java não precisou de nenhuma alteração. A extensão automatizou toda a infraestrutura do RAG para nós.



APROFUNDANDO NA ESTRATÉGIA DE DIVISÃO (SPLITTING)

APROFUNDANDO NA ESTRATÉGIA DE DIVISÃO (SPLITTING)

- ❑ Como vimos em aulas anteriores, a forma como dividimos os documentos é uma decisão de engenharia com trade-offs diretos em performance, custo e, mais importante, na qualidade da resposta.
- ❑ O EasyRAG nos permite controlar os dois parâmetros fundamentais do DocumentSplitter via application.properties:
 - ❑ `quarkus.langchain4j.easy-rag.max-segment-size`: O tamanho máximo de cada chunk em tokens.
 - ❑ `quarkus.langchain4j.easy-rag.max-overlap-size`: O número de tokens que se sobrepõem entre chunks consecutivos.

OTIMIZANDO O MAX-SEGMENT-SIZE

- ❑ O Trade-Off do Tamanho do Segmento
- ❑ Este parâmetro define a "granularidade" do nosso conhecimento. A escolha do valor ideal depende da natureza dos seus documentos.
 - ❑ Valores Pequenos (ex: 100-200 tokens):
 - ❑ Prós: Geram chunks com alta densidade semântica. Se um chunk é recuperado, é muito provável que todo o seu conteúdo seja relevante. Reduz o "ruído" enviado ao LLM.
 - ❑ Contras: Risco de perder o contexto mais amplo. Uma resposta pode exigir a combinação de múltiplos chunks, e informações que conectam ideias podem ser perdidas se estiverem em chunks separados.
 - ❑ Valores Grandes (ex: 500-1000 tokens):
 - ❑ Prós: Preservam melhor o contexto geral de uma seção do documento.
 - ❑ Contras: Risco de diluição semântica. O chunk pode conter a resposta, mas também muitos parágrafos irrelevantes, confundindo o LLM. Aumenta o número de tokens enviados a cada chamada, impactando custo e latência.
- ❑ Sugere-se iniciar com um valor de 300 tokens e validar os resultados. Para documentos mais estruturados como manuais ou termos de serviço, valores menores podem ser mais eficazes.

OTIMIZANDO O MAX-SEGMENT-SIZE

- ❑ Ajustando o application.properties:

```
# Vamos reduzir o tamanho para criar chunks mais focados.  
quarkus.langchain4j.easy-rag.max-segment-size=100
```


OTIMIZANDO O MAX-OVERLAP-SIZE

- ❑ A "Costura" Semântica entre os Chunks
- ❑ Este parâmetro previne a perda de contexto nas bordas dos chunks.
- ❑ Por que a Sobreposição é Crucial?
 - ❑ Imagine que uma frase ou parágrafo que conecta duas ideias importantes é dividido exatamente ao meio pelo splitter. Sem sobreposição, nenhuma busca que dependa da frase completa terá sucesso. A sobreposição garante que a frase completa exista em pelo menos um dos chunks, preservando a continuidade semântica.
- ❑ Como Definir?
 - ❑ Uma boa regra prática é definir a sobreposição como 10-20% do max-segment-size. Uma sobreposição muito pequena pode não ser suficiente, enquanto uma sobreposição muito grande cria redundância excessiva e aumenta o tamanho total do EmbeddingStore.

OTIMIZANDO O MAX-OVERLAP-SIZE

- ❑ Ajustando o application.properties:

```
# Vamos usar 20% do nosso max-segment-size de 100.  
quarkus.langchain4j.easy-rag.max-overlap-size=20
```

VISUALIZANDO A INGESTÃO

VISUALIZANDO A INGESTÃO

- ❑ A Dev UI do Quarkus nos permite inspecionar o resultado do nosso pipeline de ingestão.
- ❑ Passo 1: Inicie a Aplicação em Modo de Desenvolvimento
 - ❑ Com as novas propriedades configuradas, execute: `mvn quarkus:dev`
- ❑ Passo 2: Acesse a Dev UI e o Embedding Store
 - ❑ Abra seu navegador em `http://localhost:8080/q/dev`.
 - ❑ No painel, localize o card "LangChain4j Core" e clique no link "Embedding store".
- ❑ Passo 3: Analise os Chunks
 - ❑ A interface exibirá uma pesquisa para visualizar os chunks no `InMemoryEmbeddingStore`.
 - ❑ O que observar?
 - ❑ Conteúdo: Verifique o conteúdo de cada chunk. Eles fazem sentido como unidades de informação independentes?
 - ❑ Sobreposição: Você deve ver textos se repetindo, confirmando que a sobreposição está funcionando.
 - ❑ Tamanho: Embora não mostre o número de tokens, você pode ter uma noção visual se os chunks estão menores e mais focados do que antes.

RESUMO E PRÓXIMOS PASSOS

RESUMO E PRÓXIMOS PASSOS

- ❑ Dissecamos o funcionamento interno do EasyRAG, entendendo como ele automatiza a criação do InMemoryEmbeddingStore e a injeção do RetrievalAugmentor.
- ❑ Aprofundamos na engenharia da fase de ingestão, tratando a configuração do DocumentSplitter (max-segment-size e max-overlap-size) como uma decisão de design com trade-offs claros.
- ❑ Utilizamos a Dev UI do Quarkus como uma ferramenta de depuração para validar nossa estratégia de ingestão e inspecionar o estado do nosso EmbeddingStore.
- ❑ Na sequência:
 - ❑ O InMemoryEmbeddingStore nos serviu bem para desenvolvimento, mas ele possui dois problemas para um ambiente de produção:
 - ❑ Volatilidade: Todos os embeddings são perdidos a cada reinicialização da aplicação, exigindo um caro processo de re-ingestão a cada deploy.
 - ❑ Escalabilidade Limitada: O armazenamento é limitado pela memória RAM de uma única instância da aplicação.
 - ❑ Então, vamos resolver isso. Entraremos no mundo dos Vector Databases, entendendo por que eles são a espinha dorsal de sistemas RAG de produção e explorando as opções disponíveis no ecossistema Langchain4j.

ATÉ A PRÓXIMA!